

WinAPI Coop Survivor — 기술 아키텍처 및 구현 문서

개요: WinAPI + Direct2D 1.1 기반으로 직접 제작한 2D 게임 엔진 위에 구현된 최대 4인 협동 멀티플레이어 로그라이크 탐다운 슈터 게임. 상용 엔진(Unity/Unreal) 없이 C++20으로 엔진 코어부터 게임 로직, 물리, 멀티플레이 네트워크까지 직접 설계·구현

[목차]

- 프로젝트 개요 및 기술 스택
- 엔진 구성 — EngineKernel과 Manager 생명주기 (2.1 진입점 및 프레임 루프 / 2.2 매니저 등록 순서와 실행 의존성)
- 객체 생명주기 — GameObject, Component, Scene (3.1 컴포넌트 생명주기 및 지연 처리 / 3.2 다형적 복제를 위한 CLONEABLE_COMPONENT 매크로와 오프셋 재계산 / 3.3 ObserverPtr 스마트 참조 / 3.4 __declspec(property) 편의성)
- 네트워크 구현 — 결정론 한계 극복과 Host-Authoritative 전환 (4.1 락스텝 한계 / 4.2 스냅샷 및 클라이언트 최적화 / 4.3 패킷 프로토콜 명세)
- 게임플레이 핵심 시스템 (5.1 범용 카테고리 풀링 스킴 시스템 / 5.2 협동 관(Coffin) 부활 시스템)
- 초기 계획 대비 최종 결과물 비교
- 기술적 학습 및 개발 회고 (Lessons Learned)
- AI 활용 및 엔지니어링 인사이트 (AI-Assisted Engineering)

1. 프로젝트 개요 및 기술 스택

구분	기술 스택 및 라이브러리	적용 목적 및 특징
플랫폼 & 언어	Windows 10/11, C++20, WinAPI	저수준 윈도우 프로그래밍 및 고성능 게임 루프 구현
렌더링	Direct2D 1.1 + DirectWrite	하드웨어 가속 2D 스프라이트, 텍스트 및 9-Slice UI 렌더링
물리 엔진	Box2D 3.0	고정 물리 틱(FixedUpdate) 기반 강체 시뮬레이션
네트워크	UDP Socket (Winsock2)	백그라운드 수신 스레드, Host 주도 연산
GUI & 에디터	ImGui (Docking Branch)	인스펙터, 씬 하이어라키, 리소스 브라우저 및 디버그 오버레이
데이터 직렬화	nlohmann/json	씬, 프리팹, ScriptableObject(SO) 에셋 포맷 관리

2. 엔진 구성 — EngineKernel과 Manager 생명주기

2.1 진입점 및 프레임 루프 (GameApp & EngineKernel)

GameApp은 WinAPI 윈도우 생성, 메시지 디스패치(WndProc) 및 COM 라이브러리 초기화를 수행. EngineKernel은 등록된 모든 매니저 시스템(ISystem)의 프레임 진행을 총괄하는 중앙 시스템. Interface Segregation Principle(인터페이스 분리 원칙)에 따라 시스템을 자동 분류하여 최적의 순서로 실행:

```
// EngineKernel.cpp - 고정 물리 스텝과 가변 렌더링 프레임의 분리
void EngineKernel::ProcessFrame()
{
    TimeManager* pTime = TimeManager::GetInstance();
    float dt = pTime->GetDeltaTime();
    float fixedDt = pTime->GetFixedDeltaTime();

    // 1. FixedUpdate: Accumulator 기반 고정 물리/네트워크 틱 보장 (Play 상태)
    if (m_PlayState == EnginePlayState::Play) {
        while (pTime->AccumulateTime()) {
            for (auto* sys : m_vUpdatableSystems) sys->FixedUpdate(fixedDt);
            pTime->ConsumeFixedTick();
        }
    }

    // 2. Variable Update: 프레임 가변 로직 처리
    for (auto* sys : m_vUpdatableSystems) sys->Update(dt);
    for (auto* sys : m_vUpdatableSystems) sys->LateUpdate(dt);

    // 3. Render Pipeline: 렌더 전처리, 본 렌더링, 후처리 분리
    for (auto* sys : m_vRenderableSystems) sys->PreRender();
    for (auto* sys : m_vRenderableSystems) sys->Render();
    for (auto* sys : m_vRenderableSystems) sys->PostRender();

    // 4. PostFrame: 프레임 종료 후 지연 생성/삭제 큐 일괄 정리
    for (auto* sys : m_vAllSystems) sys->PostFrame();
}
```

2.2 매니저 등록 순서와 실행 의존성

GameApp::RegisterManagers()에서 매니저를 등록할 때, 등록 순서가 곧 프레임 실행 순서가 됨. 데이터 흐름에 맞춰 명확한 단방향 의존성을 유지하도록 배치.

```
[기초/입력] TimeManager → InputManager → EventBus → PoolManager
↓
[에셋/네트워크] ResourceManager → DataManager → NetworkManager
↓
[게임 로직] ActionManager → SceneManager
↓
[물리/뷰] PhysicsManager → CameraManager
↓
[렌더링] GraphicManager → RenderSystem → DebugManager → GUISystem
```

3. 객체 생명주기 — GameObject, Component, Scene

3.1 컴포넌트 생명주기 및 지연 처리(Deferred Queue)

컴포넌트는 Unity 스타일의 일관된 라이프사이클(Awake → Start → Update → LateUpdate → OnDestroy)을 따름.

- 지연 생성/삭제 메커니즘: Scene::CreateGameObject() 또는 Destroy() 호출 시 컨테이너 순회 도중 컬렉션이 훼손되는 문제를 방지하기 위해, 즉시 삭제하지 않고 m_vCreationQueue, m_vDestroyQueue에 쌓아둔 뒤 프레임 맨 끝인 PostFrameCleanup() 시점에 일괄 처리.
- 초기화 플래그 보장: m_bHasAwoken, m_bHasStarted 플래그를 통해 객체가 활성화된 첫 프레임에서만 정확히 1회 호출됨을 보장.

3.2 다형적 복제를 위한 CLONEABLE_COMPONENT 매크로와 오프셋 재계산

프리팹(Prefab) 인스턴스화 및 오브젝트 풀링 시 컴포넌트의 다형적 복제(Polymorphic Clone)가 필수적임. 수십 종의 컴포넌트에 가상 함수를 반복 작성하는 로직을 매크로 함수로 해결.

```
#define CLONEABLE_COMPONENT(Type) \
    virtual Component* Clone() override { return new Type(*this); }
```

[핵심 기술: Inspector 프로퍼티 주소 오프셋 자동 보정]

컴포넌트 복사 생성자 호출 시, 에디터 Inspector에 등록된 ExposedProperty의 포인터(void* data)가 원본 객체의 메모리 주소를 여전히 가리키는 문제가 발생. 이를 해결하기 위해 원본과 복제본 간의 메모리 거리(offset)를 계산하여 포인터를 재매핑.

```
Component::Component(const Component& other)
: m_pOwnerGameObject(nullptr), m_pTransform(nullptr), m_bIsEnabled(other.m_bIsEnabled)
{
    // 원본 인스턴스와 복제된 인스턴스 간 메모리 거리(offset) 계산
    intptr_t offset = reinterpret_cast<intptr_t>(this) - reinterpret_cast<intptr_t>(&other);

    m_vProperties = other.m_vProperties;
    for (auto& prop : m_vProperties) {
        if (prop.data != nullptr) {
            // 새 인스턴스의 멤버 주소로 자동 재계산
            prop.data = reinterpret_cast<void*>(reinterpret_cast<intptr_t>(prop.data) + offset);
        }
    }
}
```

3.3 ObserverPtr — 땀글링 포인터 방지 스마트 참조

몬스터 사망이나 투사체 파괴 등으로 타깃이 소멸했을 때, 다른 컴포넌트가 해당 객체를 계속 가리키다 발생하는 Use-After-Free 크래시를 방지하기 위해 경량 옴저버 포인터를 구축.

- ObserverPtr가 생성될 때 대상 GameObject의 옴저버 리스트에 자기 주소(void**)를 등록.
- GameObject가 소멸할 때 등록된 모든 옴저버 포인터를 nullptr로 일괄 초기화.
- std::weak_ptr의 제어 블록 오버헤드 없이 제로에 가까운 비용으로 메모리 안전성을 확보.

```
template <typename T>
inline void ObserverPtr<T>::Register() {
    if (m_ptr) {
        if constexpr (std::is_same_v<std::remove_cv_t<T>, GameObject>) {
            reinterpret_cast<GameObject*>(m_ptr)->RegisterObserverPtr(reinterpret_cast<void**>(&m_ptr));
        }
    }
}
```

3.4 __declspec(property) 기반 문법적 편의성

MSVC의 __declspec(property) 확장을 적용하여 함수 호출 괄호 없이 멤버 변수처럼 직관적으로 접근하면서도, 내부적으로 널 참조 방지 assert를 수행하도록 캡슐화.

```
__declspec(property(get = GetGameObjectInternal)) GameObject& gameObject;
__declspec(property(get = GetTransformInternal)) TransformComponent& transform;

// 사용 코드 예시:
transform.SetPosition(newPos);
gameObject.SetActive(false);
```

4. 네트워크 구현 — 결정론 한계 극복과 Host-Authoritative 전환

4.1 아키텍처 전환 배경: 결정론적 시뮬레이션(Lockstep)의 한계

초기 프로토타입에서는 락스텝 방식을 채택했으나 다음과 같은 현실적 한계가 존재함:

1. 부동소수점 오차 누적: x64 플랫폼과 컴파일러 최적화 옵션에 따라 미세한 부동소수점 차이가 발생하여 시간이 지남에 따라 몬스터 위치가 완전히 발산(Desync)함.
2. Box2D 물리 비결정성: 패킷 지터로 인해 충돌 프레임 순서가 1틱만 달라져도 튕겨 나가는 반발 궤적이 클라이언트마다 완전히 달라짐.
3. 조작감 저하 (Input Lag): 4명의 플레이어 입력을 모두 기다리는 구조로 인해 핑이 높은 유저가 있으면 모든 유저의 조작이 멈추는 현상 발생.

4.2 Host-Authoritative 스냅샷 및 최적화

Host가 게임 상태의 유일한 권위자(Authority)로 동작하고, Client는 입력만 전달하며 상태 스냅샷을 수신해 보관.

1. Client 몬스터의 Kinematic Sensor 전환: Client 환경에서는 몬스터의 Box2D 설정을 Kinematic Body 및 Sensor로 전환하여 로컬 물리 충돌 연산을 완전히 배제하고 오직 렌더링 보관에만 집중.
2. Soft Correction Velocity (부드러운 위치 보정): Host로부터 수신한 위치와 Client 로컬 위치 사이에 오차가 발생했을 때 순간이동(Snap)시키지 않고, 거리 오차에 비례하는 보정 속도 벡터를 부여하여 자연스럽게 보정.
3. 비동기 I/O 전용 스레드: recvfrom() 블로킹으로 인한 메인 루프 지연을 방지하기 위해 백그라운드 수신 스레드를 운영하며, 뮤텍스 보호 큐를 통해 FixedUpdate 틱마다 일괄 패킷 소비.

4.3 핵심 패킷 프로토콜 명세

#pragma pack(push, 1) 기반으로 패딩 없이 압축된 바이너리 패킷 사용.

패킷 타입	방향	동기화 주기	설명
CLIENT_CONN_REQ / RES	C ↔ H	이벤트	접속 요청, 정원 확인 및 NetID 할당
PLAYER_INPUT	C → H	60 Hz	클라이언트 플레이어 위치, 속도, 조준 각도
MONSTER_SNAPSHOT	H → C	20~30 Hz	활성 몬스터 일괄 스냅샷 (NetID, AssetID, Position)
SKILL_FIRE / SLOT_SYNC	H ↔ H	이벤트	투사체 발사 동기화, 레벨업 스킬 슬롯/레벨 브로드캐스트
TEAM_EXP_SYNC / PARTY_HP	H → C	가변 주기	공유 팀 경험치/레벨, 전 파티원 실시간 HP 동기화
MONSTER_KILL	H → C	이벤트	몬스터 사망 및 경험치 보석(ExpGem) 드롭 위치
GAME_OVER_SIGNAL	H → C	이벤트	전원 사망 시 게임오버 씬 전환 트리거

5. 게임플레이 핵심 시스템

5.1 범용 카테고리 기반 풀링 스킬 시스템

- 데이터 주도 설계 (SkillSO): 기획 데이터(공격력, 쿨다운, 투사체 속도, 관통 횟수 등)를 에셋화하여 코드 수정 없이 인스펙터에서 튜닝 가능.
- 3대 카테고리 일반화:
 - ProjectileComponent: 방향성 투사체 (직선, 유도, 도끼 포물선 등)
 - AoEComponent: 범위 지속/단발 폭발 피해
 - AuraComponent: 플레이어 중심 회전 및 지속 오라 피해
- 모든 스킬 인스턴스는 PoolManager를 통해 사전 생성되어 GC/할당 오버헤드 배제.

5.2 협동 관(Coffin) 부활 시스템

- 플레이어 HP가 0이 되면 즉시 게임오버가 되지 않고 Coffin(관) 프리팹으로 전환.
- 생존한 다른 플레이어가 관 주변 반경에 진입하여 인터랙션 키를 유지하면 진행 바 UI가 채워지며 부활.
- 네트워크 패킷(PARTY_HP_SYNC)과 연동되어 파티원 전원이 사망했을 때만 GameOver 씬으로 전환.

6. 초기 계획 대비 최종 결과물 비교

항목	초기 기획 및 프로토타입	최종 완성 아키텍처 및 구현 결과물
엔진 구조	기본 WinAPI 메시지 루프 기반 단일 파일 구조	EngineKernel 중앙 조율, Component-Entity 계층 구조 및 ImGui 통합 자체 엔진 완성
동기화 방식	결정론적 락스텝(Lockstep) 시뮬레이션	Host-Authoritative 상태 스냅샷 + Client Kinematic Sensor + Soft Correction 속도 보정
스킬 시스템	스킬별 개별 하드코딩 구현	ScriptableObject 데이터 에셋화 + 범용 3종 카테고리 폴링 구조 구축
협동 메카닉	단순 멀티플레이 접속 및 동시 공격	팀 공유 레벨/경험치 시스템, 실시간 파티 체력 HUD, 관(Coffin) 상호작용 부활 메카닉
에디터 도구	코드 상 하드코딩된 좌표 배치	ImGui 인스펙터, 하이여라키, 씬 저장/로드, 프리팹 시스템 및 9-Slice UI 렌더링

7. 기술적 학습 및 개발 회고 (Lessons Learned)

1) 엔진 생명주기 및 지연 큐(Deferred Queue) 설계의 중요성 체득

프레임 루프 순회 도중 객체가 생성/삭제될 때 발생하는 컨테이너 반복자 무효화(Iterator Invalidation) 문제를 겪으며, 모든 생성/파괴 요청을 지연 큐에 적재하고 PostFrame 시점에 일괄 처리하는 패턴의 안정성을 확인.

2) 메모리 레이아웃 이해와 커스텀 스마트 참조 패턴 구축

C++ 복사 생성 시 얇은 복사로 인한 댕글링 포인터 문제를 메모리 오프셋 계산($offset = this - \&other;$)을 통해 해결. 또한 `std::weak_ptr`의 무거운 컨트롤 블록 대신 소멸자 기반의 경량 `ObserverPtr`를 자체 구현하여 Use-After-Free 크래시를 원천 차단하는 경험 획득.

3) 네트워크 이론과 실제 환경 간의 간극 극복

결정론적 락스텝 방식이 부동소수점 오차 및 물리 엔진 누적 오차로 인해 실시간 액션 게임에서 쉽게 발산함을 확인. 현실적인 대안으로 Host-Authoritative 상태 스냅샷을 도입하고 클라이언트 몬스터를 Kinematic Sensor로 전환함으로써 안정적인 멀티플레이 환경을 구축하는 아키텍처적 유연성을 학습.

8. AI 활용 및 엔지니어링 인사이트 (AI-Assisted Engineering)

8.1 AI 활용을 통한 개발 생산성 혁신

- 정보 탐색 시간의 획기적 단축: 기존에 Google 검색, StackOverflow, MSDN 문서를 장시간 탐색해야 했던 WinAPI/Direct2D/Box2D 세부 API 규격 및 디자인 패턴 참조 시간을 극적으로 단축.
- 수학적 연산 구현 가속: 패킷 직렬화 구조체, 다양한 투사체 궤적 계산(포물선/유도 삼각함수), 9-Slice 정점 좌표 연산 등 기능 구현 단계에서의 생산성을 대폭 향상.

8.2 AI 활용 과정에서의 한계와 엔지니어링 검증의 필요성

- 할루시네이션(환각) 및 세부 빌드 맥락 결여: 존재하지 않는 API 호출이나 Precompiled Header(pch.h)에 이미 선언된 헤더를 개별 파일에 중복 include하는 등 C++ 컴파일/링킹 레벨의 디테일 오류가 발생하여 최종 코드 검증이 필수적임.
- 질문자 맥락에 종속적인 해결책 제시 한계: 개발자가 구조적 키워드나 명확한 아키텍처 제약을 제시하지 않을 경우, 장기적 유지보수성보다 당장 눈앞의 오류만 해소하는 근시안적 코드를 제시하는 경향 확인.
- 결론: 아키텍처 설계와 최종 코드 검증은 프로그래머가 주도하고, 기능 구현 및 정보 탐색 등은 AI를 활용해서 생산성 향상.